Simplified FlashAttention on Modern GPUs: Reproducing FlashAttention-1 and Isolating a Key FA2 Forward Idea

Calder Katyal
Yale University
CPSC 4240: Parallel Programming
calder.katyal@yale.edu

Devon Sawyer
Yale University
CPSC 4240: Parallel Programming
devon.sawyer@yale.edu

Akshat Badgamia
Yale University
CPSC 4240: Parallel Programming
akshat.badgamia@yale.edu

Abstract

FlashAttention is the standard exact attention kernel used in modern large language models because it substantially reduces high-bandwidth-memory traffic while preserving exact attention outputs. In this project, we implement a compact CUDA C++ attention benchmark suite centered on the most important broad ideas from FlashAttention-1: shared-memory tiling, online softmax, fused score/softmax/value computation, mixed precision, and tensor-core score tiles. We then add one carefully scoped extension motivated by later versions: a split-KV sequence-parallel path inspired by an important forward-only implementation idea in FlashAttention-2. Our goal is not to reproduce the full production codebase; rather, it is to isolate which ideas are necessary to outperform naive attention on modern CUDA hardware and to measure how much additional benefit is provided by the FA2-style modification once the FlashAttention-1 core is already in place. The resulting benchmark suite compares a naive mixed-precision baseline, a simplified FlashAttention-1 kernel, a FlashAttention-2-inspired forward extension, ablations of the most important FA1 design decisions, and two official baselines: FlashAttention-1 and FlashAttention-2.

1 Introduction and Motivation

Self-attention is a central primitive in transformer models, but the naive implementation is expensive in both computation and memory traffic. Given queries $Q \in \mathbb{R}^{N\times d}$, keys $K \in \mathbb{R}^{N\times d}$, and values $V \in \mathbb{R}^{N\times d}$, standard attention computes

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V. \tag{1}$$

The bottleneck is not only the $O(N^2)$ arithmetic from the score matrix $QK^\top$, but also the cost of writing and rereading the full $N \times N$ matrix from high-bandwidth memory. FlashAttention addresses this by reorganizing exact attention so that score computation, normalization, and value accumulation are fused over tiles, avoiding explicit materialization of the full attention matrix (**?**).

Our project goal is narrower than reimplementing the entire official repository. We instead aim to answer three concrete questions.

1. Which FlashAttention-1 ideas are most necessary for a meaningful modern GPU benchmark?
2. Once a simplified FlashAttention-1 kernel is implemented, does a key forward-only idea from FlashAttention-2 still help?
3. How large is the gap between a compact educational CUDA implementation and the production `flash_attn` module?

To answer these questions, we implement a GPU-only benchmark suite with a naive mixed-precision baseline, a simplified FlashAttention-1 core, one later-version-inspired extension, and two ablations that remove key FA1 ingredients. The benchmark suite is designed to compare these project kernels against both official FlashAttention-1 and official FlashAttention-2 on the same CUDA machine.

2 Background

2.1 Attention and the memory bottleneck

For a single head, standard attention computes three conceptual stages: score formation, row-wise normalization, and value mixing. In matrix form,

$$S = \frac{QK^\top}{\sqrt{d}}, \tag{2}$$

$$P = \mathrm{softmax}(S), \tag{3}$$

$$O = PV. \tag{4}$$

The mathematical definition is simple, but the straightforward implementation is expensive on GPUs because it repeatedly reads and writes the full $N \times N$ score matrix. This cost becomes especially severe at long sequence length, where the score matrix dominates both memory traffic and intermediate storage. FlashAttention's central insight is that exact attention can be reorganized around the GPU memory hierarchy: instead of storing the full score matrix, it computes attention tile-by-tile while carrying only the sufficient per-row normalization statistics needed to preserve exactness (**?**).

2.2 Why online softmax gives exact attention

The key algebraic trick behind FlashAttention-1 is that softmax normalization can be updated incrementally across score tiles without approximation. Suppose a row has already processed some subset of keys and currently stores a running maximum m and a running denominator $\ell = \sum_j e^{s_j - m}$. When a new tile of scores arrives, let $\hat{m}$ be the maximum score inside that tile and define the new row maximum as $m' = \max(m, \hat{m})$. Then the updated denominator can be written exactly as

$$\ell' = e^{m-m'}\ell + \sum_{j \in \text{new tile}} e^{s_j - m'}. \tag{5}$$

The same rescaling factor $e^{m-m'}$ can be applied to the running output accumulator before adding the new tile's contribution. This recurrence is what lets FlashAttention remain exact while discarding the old score tile once its contribution has been folded into the running state (**?**). In other words, the algorithm reduces the memory footprint by changing the order of computation, not by changing the mathematical attention rule.

2.3 Evolution from FlashAttention-1 to FlashAttention-4

The FlashAttention family has evolved in a way that is useful for framing our project. FA1 contributes the core IO-aware exact-attention reformulation. FA2 improves forward performance through better work partitioning and reduced overhead beyond raw matrix multiplication (**?**). FA3 is much more architecture-specific: its key contributions are asynchronous execution, warp specialization, and deeper co-design with Hopper hardware (**?**). FA4 pushes this hardware/software co-design further for asymmetric scaling on newer systems, emphasizing redesigned fully asynchronous MMA pipelines, larger tiles, and reduced non-matmul bottlenecks such as exponential evaluation (**?**). These later versions are important context for understanding the trajectory of the family, even though our

implementation target remains a compact FA1-centered kernel plus a single FA2-inspired forward extension.

This scoping decision matches our experimental goal. We are not training models, so we prioritize the features that most strongly affect forward-pass performance:

- no explicit score-matrix materialization,
- shared-memory tiling,
- online softmax,
- fused exact attention,
- mixed precision, and
- tensor-core-friendly score computation.

These are the parts of FlashAttention that are both conceptually central and broadly relevant on modern NVIDIA GPUs.

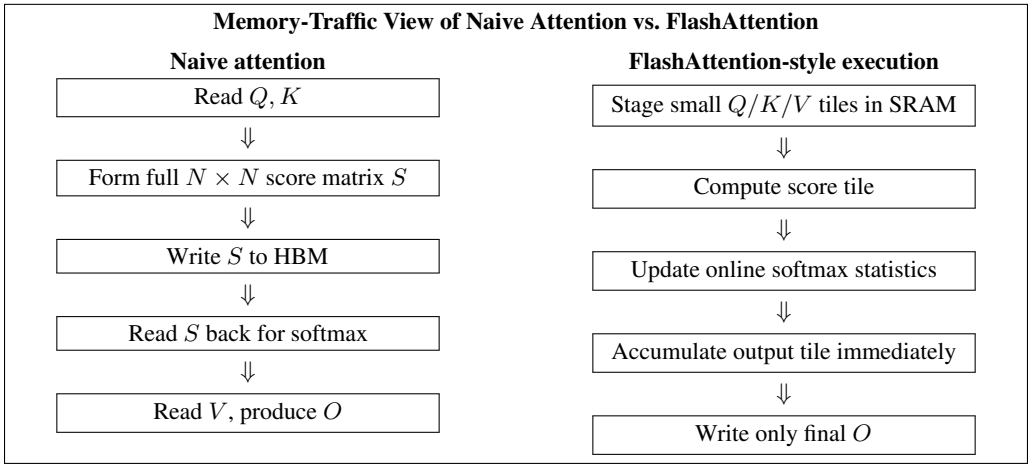

Figure 1: The core memory-efficiency idea of FlashAttention. The naive implementation repeatedly touches the full score matrix in high-bandwidth memory, whereas FlashAttention keeps only tiles and per-row running statistics live at any moment.

3 Methods

3.1 Naive mixed-precision baseline

Our baseline uses the usual three-stage attention decomposition: compute $QK^\top$, apply row-wise softmax, and finally multiply by V. The implementation uses FP16 inputs so that it is compared against the project kernels under similar data-type assumptions, but it still materializes the full score matrix in FP32. This baseline therefore preserves exact attention math while exposing the major memory-system weakness that FlashAttention is designed to remove.

3.2 Simplified FlashAttention-1 kernel

The main project kernel is our simplified FlashAttention-1 implementation. It stages query, key, and value tiles in shared memory, computes score tiles with WMMA tensor-core fragments, and maintains per-row running softmax statistics across KV tiles. Let m_i and ℓ_i denote the running row maximum and running normalization term for query row i. For a new KV tile producing scores s_{ij}, the online

recurrence is

$$m'_i = \max\left(m_i, \max_j s_{ij}\right), \tag{6}$$

$$\ell'_i = e^{m_i - m'_i}\ell_i + \sum_j e^{s_{ij} - m'_i}, \tag{7}$$

$$o'_i = e^{m_i - m'_i} o_i + \sum_j e^{s_{ij} - m'_i} V_j. \tag{8}$$

After the final tile, each output row is normalized by ℓ'_i. This is the main algorithmic reason FlashAttention can remain exact while avoiding the full $N \times N$ attention matrix (**?**).

Our kernel is intentionally smaller than the official implementation, but it preserves the core forward-pass structure:

- query, key, and value tiles are staged in shared memory;
- the score tile is accumulated with tensor-core matrix-multiply-accumulate operations;
- softmax normalization and value accumulation are fused into the same kernel; and
- only per-row statistics and the final output are written to global memory.

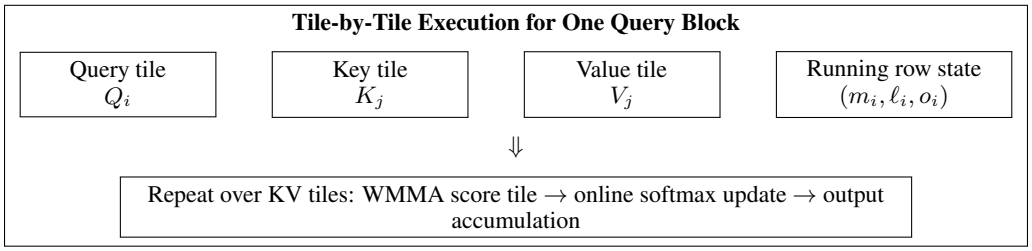

Figure 2: Conceptual per-tile execution order. The output row is built incrementally as KV tiles are processed, which eliminates the need to materialize the full attention matrix.

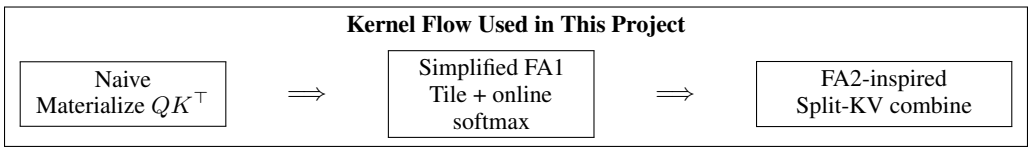

Figure 3: Conceptual progression of the benchmarked methods. The naive baseline and the two project kernels all compute exact causal attention, but they differ in how work and memory movement are organized.

3.3 FA2-inspired extension: split-KV sequence parallelism

The most important portable later-version idea we add is a split-KV forward path. Rather than assigning one query tile to a single block that scans the full KV range, we partition the KV tiles across multiple blocks. Each block computes partial statistics for a subset of the KV tiles assigned to the same query tile, producing partial maxima $m_i^{(r)}$, partial normalizers $\ell_i^{(r)}$, and partial output accumulators $o_i^{(r)}$ for split r. These are then combined exactly with

$$m_i = \max_r m_i^{(r)}, \tag{9}$$

$$\ell_i = \sum_r \ell_i^{(r)} e^{m_i^{(r)} - m_i}, \tag{10}$$

$$o_i = \frac{1}{\ell_i} \sum_r o_i^{(r)} e^{m_i^{(r)} - m_i}. \tag{11}$$

This extension is motivated by the upstream split-KV path and its `num_splits` heuristic in the official implementation. It is not the entire FlashAttention-2 redesign, but it does capture one central implementation idea that remains meaningful outside a particular model-serving stack (**?**).

3.4 Ablations

We include two ablation kernels so that the report can answer which FA1 ingredients matter most.

- **No online softmax.** This kernel keeps tiling and tensor-core score tiles, but replaces the online recurrence with a two-pass computation.
- **No SRAM tiling.** This kernel keeps the exact online-softmax logic but loads key/value rows directly from global memory instead of shared-memory tiling.

The ablations are important because they let us isolate whether the main gain comes from the online recurrence, from the memory-hierarchy reorganization, or from both together.

Table 1: What each implemented method is meant to test.

Method	Upstream concept tested	Exact?	Primary question answered
Naive	Standard attention decomposition	yes	What is the baseline cost when the score matrix is fully materialized?
Simplified FA1	Tiling + online softmax + fusion	yes	How much of the main FlashAttention win appears once the core FA1 ideas are implemented?
FA2-inspired	Split-KV sequence parallelism	yes	Does KV-range partitioning help after the FA1 core is already in place?
Ablations	Removal of one FA1 ingredient at a time	yes	Which FA1 ingredients are actually carrying the performance gain?

3.5 Why we use an FA2-inspired split-KV kernel

The FA2-style idea we chose is split-KV sequence parallelism. This choice is deliberate: it is a real forward-only mechanism from the FlashAttention codebase, it is important enough to matter for performance, and it can be isolated cleanly in a student implementation. In contrast, many of the other FA2 improvements are more entangled with larger kernel-engineering changes. Split-KV therefore gives us the best tradeoff between faithfulness to the official implementation and interpretability in the final benchmark.

3.6 Implementation details relevant to the GPU study

Two implementation choices are especially important for interpreting the benchmark. First, all project kernels use FP16 inputs and FP32 accumulation. This keeps the benchmark aligned with the mixed-precision regime used by modern FlashAttention implementations while avoiding the additional complexity of BF16 support in our student codebase. Second, the head dimension is fixed to $d = 64$ for the main benchmark sweep. This is small enough to be common in practice, but also aligned with our WMMA score path and with the tensor-core-friendly restriction that d be a multiple of 16.

The simplified FA1 kernel and the FA2-inspired kernel also share most of the same surrounding choices: both perform exact causal attention, both keep the output accumulation in FP32, and both use the same correctness reference in the benchmark binary. This is important because it means the FA1-versus-FA2 comparison is not confounded by datatype changes, approximation, or a different masking convention. The intended interpretation is therefore narrow and clean: if FA2-inspired split-KV helps, it should help because of additional sequence-level parallelism and altered work partitioning, not because the kernels are doing different mathematical tasks.

3.7 Kernel layout

Our CUDA implementation uses a small tile size of 16×16, which aligns naturally with WMMA fragments and keeps the kernels easy to reason about. For the simplified FA1 kernel, one query tile is

assigned to a block. The query tile, a transposed key tile, and the corresponding value tile are staged in shared memory; the score tile is formed with tensor cores; and each query row maintains a running maximum, running normalization constant, and running output vector in FP32. Once all KV tiles have been processed, the output accumulator is normalized and written back to global memory.

The FA2-inspired kernel keeps the same tile-local computation but changes how the KV range is traversed. Instead of assigning the full KV range to one block, it partitions the KV tiles into splits. Each split produces partial m, ℓ, and o values, and a second kernel combines them exactly. This split/combine structure is the main implementation difference between our FA1 and FA2-inspired forward paths.

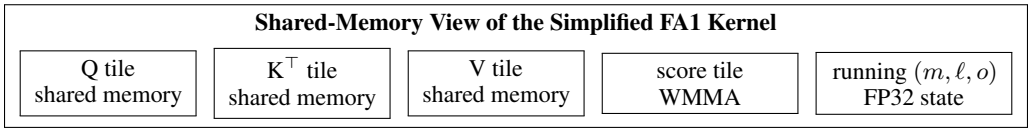

Figure 4: Conceptual shared-memory organization for the simplified FA1 kernel. The key idea is that only the current tiles and row-wise running state must remain resident, rather than the full score matrix.

Table 2: Kernel properties used in the final project benchmark.

Method	Exact	Causal	FP16 in	FP32 accum	Tensor cores	Split-KV
Naive	yes	yes	yes	yes	no	no
Simplified FA1	yes	yes	yes	yes	yes	no
FA2-inspired	yes	yes	yes	yes	yes	yes
No online softmax	yes	yes	yes	yes	yes	no
No SRAM tiling	yes	yes	yes	yes	no	no

4 Experimental Setup

We study dense causal self-attention in a controlled forward-only setting. All methods are evaluated on the same tensor shapes and masking convention so that differences in runtime reflect kernel organization rather than changes in the mathematical task. The project kernels and the official baselines are all run in mixed precision with FP16 inputs. Output accumulation and the reference comparison are performed in FP32, which matches the numerically stable regime used throughout the FlashAttention literature (**??**).

Table 3: Benchmark configuration used by the CUDA benchmark suite.

Quantity	Value	Quantity	Value
Batch size B	2	Heads H	8
Head dimension d	64	Causal mask	yes
Input dtype	FP16	Accumulation dtype	FP32
Sequence lengths	128–4096	Warmup iters	3
Timed iters	10	Official package dtype	FP16
Head-dim constraint	$d \equiv 0 \pmod{16}$	Target hardware	modern NVIDIA GPU

The benchmark uses one fixed batch/head configuration and varies only sequence length. This makes the role of memory traffic particularly visible, since increasing N enlarges the naive score matrix quadratically while leaving the head dimension fixed. We report runtime, approximate memory footprint, and maximum absolute error relative to an FP32 reference.

The evaluation reports four classes of comparisons.

1. **Naive vs. simplified FA1.** This isolates the main FlashAttention-1 win.
2. **Simplified FA1 vs. FA2-inspired.** This isolates split-KV sequence parallelism.
3. **Project kernels vs. official FlashAttention-1 and FlashAttention-2.** This measures the gap between the educational implementation and the production kernels from both generations.

4. **FA1 ablations.** These isolate which of the core FA1 ingredients are necessary for the speedup.

The official baselines serve two distinct roles in this study. Official FlashAttention-1 anchors the comparison to the original IO-aware reformulation, while official FlashAttention-2 reflects the more mature forward kernel family that is widely used on modern NVIDIA GPUs.

5 Results

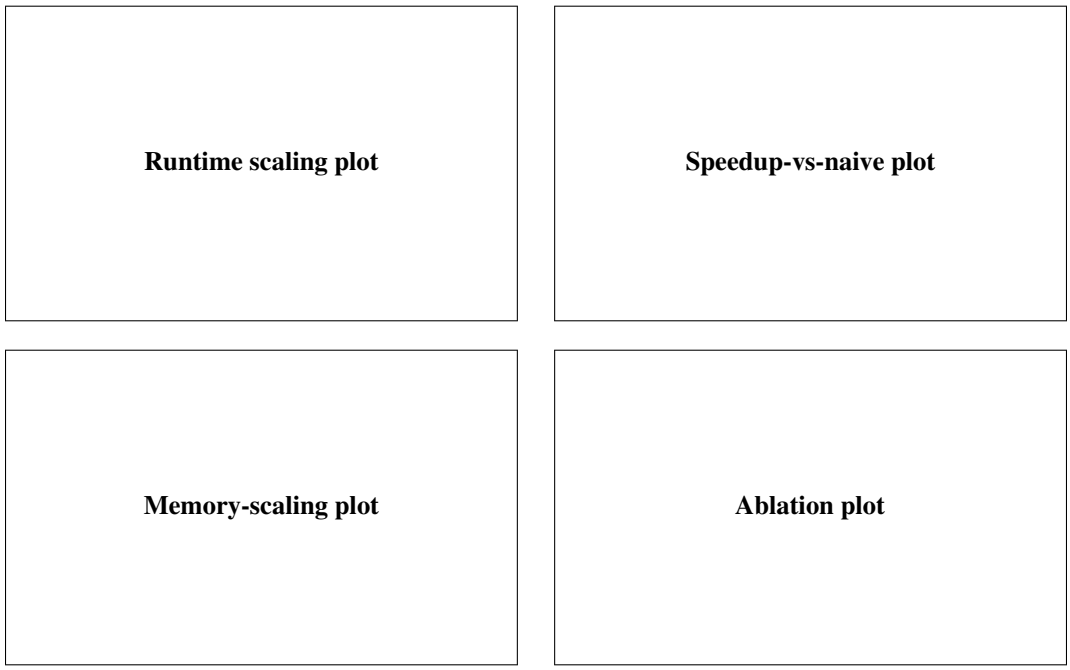

Figure 5: Runtime, speedup, memory, and ablation views for the benchmark suite, including both official FlashAttention baselines when those CSVs are present.

Table 4: Main runtime comparison. Entries report average kernel runtime in milliseconds.

Method	$N = 512$	$N = 1024$	$N = 2048$	$N = 4096$
Naive	—	—	—	—
Simplified FA1	—	—	—	—
FA2-inspired extension	—	—	—	—
Official FlashAttention-1	—	—	—	—
Official FlashAttention-2	—	—	—	—

Table 5: Speedup relative to the naive baseline. This table directly quantifies the effect of the FlashAttention-1 core and the FA2-inspired addition.

Method	$N = 512$	$N = 1024$	$N = 2048$	$N = 4096$
Simplified FA1	—	—	—	—
FA2-inspired extension	—	—	—	—
Official FlashAttention-1	—	—	—	—
Official FlashAttention-2	—	—	—	—

Table 6: Ablation table for the FlashAttention-1 core. Runtime numbers are interpreted relative to simplified FA1 at the same sequence length.

Method	Runtime at $N = 2048$ (ms)	Speedup vs. naive	Max abs. error
Simplified FA1	—	—	—
Ablation: no online softmax	—	—	—
Ablation: no SRAM tiling	—	—	—

Taken together, Figure **??** and Tables **??**–**??** answer complementary questions. The runtime table gives the most direct method-to-method comparison at representative sequence lengths, the speedup table makes the incremental effect of the FA1 core and the FA2-inspired extension easier to read relative to the naive baseline, and the ablation table identifies which FA1 ingredients are indispensable once the benchmark moves into the long-sequence regime.

6 Discussion

The evaluation is organized so that each comparison answers a specific systems question. The most important comparison is naive attention versus simplified FA1, since that isolates the gain from IO-aware tiling, online softmax, and score-matrix elimination. The FA1-versus-FA2 comparison is intentionally narrower: it asks whether one specific forward-only idea from FA2—split-KV sequence parallelism with an exact combine step—still contributes additional benefit once the FA1 core is already in place. Comparing against official FlashAttention-1 and FlashAttention-2 then provides an external reference frame for how far a compact implementation can approach the production kernels from those two generations.

7 Limitations

This project still has several deliberate limitations, all of which should be acknowledged in the final presentation of results.

- The code is forward-only and does not include training kernels.
- The kernels currently target head dimensions that are multiples of 16 and at most 128.
- The implementation uses FP16 inputs and FP32 accumulation, but does not yet support BF16.
- The FA2-inspired addition is intentionally narrow and should not be presented as a full reproduction of FA2.
- The comparison against the official FA1 and FA2 packages is numerically meaningful and practically useful, but it is not an apples-to-apples feature comparison.

These limitations are acceptable for our stated project goal because that goal is to identify the most important ideas behind FlashAttention's forward-pass speedups, not to reproduce the entire production stack.

8 Conclusion

This project studies how much of FlashAttention's practical forward-pass advantage can be captured in a compact CUDA C++ implementation. The main kernel preserves the central FlashAttention-1 design decisions: shared-memory tiling, online softmax, fusion, mixed precision, and tensor-core score computation. The additional FA2-inspired split-KV path then tests whether sequence-parallel work partitioning offers a further gain once the FA1 core is already in place. Taken together, these kernels provide a focused benchmark suite for understanding which algorithmic and hardware-conscious ideas are most responsible for FlashAttention's speedup over naive exact attention.

References

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. 2023. arXiv:2307.08691.

Jay Shah, Ganqu Cui Fu, Hazael Q. Kumbong, Konstantinos Roumeliotis, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchronous low-precision computation. In *Advances in Neural Information Processing Systems*, 2024.

Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. FlashAttention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. 2026. arXiv:2603.05451.